Cloudflare Turnstile Setup Guide — SkillSphere

> *React Native + Web / Express Backend | Free Forever | No User Friction*

What is Cloudflare Turnstile?

Cloudflare Turnstile is a free CAPTCHA replacement that protects forms from bots without annoying real users. It analyzes browser signals silently in the background — no "click the traffic lights" puzzles.

Flow: User fills form → Turnstile checks silently → Token generated → Backend verifies

> ***Key Principle:*** *Turnstile runs on the web frontend only. For Android, the token is simply not sent and the backend skips the check — Android is protected by backend rate limiting instead.*

Part 1 — Getting Your Cloudflare Keys

Step 1: Create a Cloudflare Account

- Go to `cloudflare.com` and click **Sign Up**
- Enter email and password, then verify your email
- You do **not** need to add a domain or change DNS — Turnstile works standalone

---

Step 2: Navigate to Turnstile

After logging in:

1. Look at the **left sidebar**
2. Scroll down until you see **"Turnstile"**
3. Click it — you will see the Turnstile overview page

Step 3: Add Your Site

Click **"Add site"** (top right corner) and fill in:

Field	Value
Site name	`SkillSphere` (just a label)
Domain	`skillsphere.com.pk`
Widget type	**Managed** ← select this

Widget Type Options:

Type	Behaviour	Use When
Managed	Cloudflare decides challenge vs invisible	✅ Recommended
Non-interactive	Always invisible, no challenge ever	Low-risk forms
Invisible	Invisible but can run background challenges	Middle ground

Click **Create** when done.

Step 4: Copy Your Keys

After clicking Create, Cloudflare shows two keys. **Copy both now.**

```
Site Key:   0x4AAAAAAA_xxxxxxxxxxxxxxxxxxxxxxx  →  Frontend code (safe to be public)
Secret Key: 0x4AAAAAAA_xxxxxxxxxxxxxxxxxxxxxxx  →  backend/.env ONLY (never expose)
```

> ⚠️ ***CRITICAL:*** *Never put the Secret Key in frontend code or commit it to Git.*

Step 5: Restrict Domain (Security Step)

1. Turnstile dashboard → click your site → **Settings**
2. Under **Allowed Origins / Domains**, ensure only `skillsphere.com.pk` is listed
3. Remove any wildcards `*` if present
4. Click **Save**

Part 2 — Environment Variables

Step 6: Add Secret Key to backend/.env

Open `backend/.env` and add:

```
# ... existing variables ...
TURNSTILE_SECRET_KEY=0x4AAAAAAA_your_secret_key_here
```

For development, use Cloudflare's official test key that always passes:

```
# Development only:
TURNSTILE_SECRET_KEY=1x0000000000000000000000000000000AA
```

Part 3 — Official Test Keys (Development)

Use these during development so you don't need to pass real challenges while building.

Purpose	Site Key (Frontend)	Secret Key (Backend)
✅ Always Passes	`1x00000000000000000000AA`	`1x0000000000000000000000000000000AA`
❌ Always Blocks	`2x00000000000000000000AB`	`2x0000000000000000000000000000000AA`
🔒 Always Challenges	`3x00000000000000000000FF`	`3x0000000000000000000000000000000AA`

> *Switch to real keys before deploying to production.*

Part 4 — Code Implementation

This section shows the exact changes needed in your SkillSphere codebase.

File 1: `AppAndroidSS/web/index.html`

Add the Turnstile script inside `<head>`:

```
<!-- Cloudflare Turnstile — add before </head> -->
<script src="https://challenges.cloudflare.com/turnstile/v0/api.js"
        async defer></script>
```

File 2: `AppAndroidSS/src/screens/auth/SplitAuthCard.js`

This file contains both login and signup forms (`SplitAuthCard` and `MobileAuthCard`).

2a — Add captcha token state (inside `SplitAuthCard` component, near the other state declarations at line ~102):

```
// After the existing state declarations (loginError, signupError, etc.)
const [captchaToken, setCaptchaToken] = useState('');
```

2b — Register Turnstile callbacks (inside the existing `useEffect` at line ~121, or add a new one next to it):

```
useEffect(() => {
  if (Platform.OS === 'web') {
    window.onTurnstileVerified = (token) => setCaptchaToken(token);
    window.onTurnstileExpired  = ()      => setCaptchaToken('');
    window.onTurnstileError    = ()      => setCaptchaToken('');
  }
}, []);
```

2c — Update `handlePasswordLogin` (currently at line ~151):

```
// BEFORE:
const handlePasswordLogin = async () => {
  setLoginError('');
  if (!loginEmail.trim()) return setLoginError('Please enter your email address');
  if (!loginPassword)     return setLoginError('Please enter your password');
  const result = await login(loginEmail.trim().toLowerCase(), loginPassword);
  if (!result.success) setLoginError(result.error || 'Login failed. Please check your
credentials.');
};

// AFTER:
const handlePasswordLogin = async () => {
  setLoginError('');
  if (!loginEmail.trim()) return setLoginError('Please enter your email address');
  if (!loginPassword)     return setLoginError('Please enter your password');
  if (Platform.OS === 'web' && !captchaToken) return setLoginError('Please wait for security
verification');
  const result = await login(loginEmail.trim().toLowerCase(), loginPassword, captchaToken);
  if (!result.success) setLoginError(result.error || 'Login failed. Please check your
credentials.');
};
```

2d — Update `handleSendLoginOTP` (currently at line ~159):

```
// BEFORE:
const handleSendLoginOTP = async () => {
  setLoginError('');
  if (!loginEmail.trim()) return setLoginError('Please enter your email address');
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(loginEmail.trim())) return setLoginError('Please
enter a valid email address');
  setSendingLoginOTP(true);
  try {
    const result = await sendLoginOTP(loginEmail.trim().toLowerCase());
    // ...rest unchanged

// AFTER — only add the captcha check after email validation:
const handleSendLoginOTP = async () => {
  setLoginError('');
  if (!loginEmail.trim()) return setLoginError('Please enter your email address');
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(loginEmail.trim())) return setLoginError('Please
enter a valid email address');
  if (Platform.OS === 'web' && !captchaToken) return setLoginError('Please wait for security
verification');
  setSendingLoginOTP(true);
  try {
    const result = await sendLoginOTP(loginEmail.trim().toLowerCase(), captchaToken);
    // ...rest unchanged
```

2e — Update `handleSignupSendOTP` (currently at line ~202):

```
// BEFORE:
const handleSignupSendOTP = async () => {
  setSignupError('');
  if (!signupName.trim())  return setSignupError('Please enter your full name');
  if (!signupEmail.trim()) return setSignupError('Please enter your email address');
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(signupEmail.trim())) return setSignupError('Please
enter a valid email address');
  setSendingSignupOTP(true);
  try {
    const result = await sendOTP(signupEmail.trim().toLowerCase(), signupName.trim());
    // ...rest unchanged

// AFTER — only add the captcha check after email validation:
const handleSignupSendOTP = async () => {
  setSignupError('');
  if (!signupName.trim())  return setSignupError('Please enter your full name');
  if (!signupEmail.trim()) return setSignupError('Please enter your email address');
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(signupEmail.trim())) return setSignupError('Please
enter a valid email address');
  if (Platform.OS === 'web' && !captchaToken) return setSignupError('Please wait for
security verification');
  setSendingSignupOTP(true);
  try {
```

```
    const result = await sendOTP(signupEmail.trim().toLowerCase(), signupName.trim(),
captchaToken);
    // ...rest unchanged
```

2f — Add widget to the JSX — place this just above the submit button in both login and signup form panels (search for the primary button in each panel):

```
{/* Cloudflare Turnstile widget — web only */}
{Platform.OS === 'web' && (
  <div
    className="cf-turnstile"
    data-sitekey="YOUR_SITE_KEY_HERE"
    data-callback="onTurnstileVerified"
    data-expired-callback="onTurnstileExpired"
    data-error-callback="onTurnstileError"
    style={{ marginBottom: 14 }}
  />
)}
```

> *Replace `YOUR_SITE_KEY_HERE` with `1x00000000000000000000AA` for development, your real Site Key for production.*

File 3: `AppAndroidSS/src/services/apiClient.js`

Update `authAPI` (currently at line ~199) to pass `captchaToken`:

```
// BEFORE:
export const authAPI = {
  login: (email, password) => post('/auth/login', { email, password }, false),
  // ...
  sendOTP: (email, name) => post('/auth/send-otp', { email, name }, false),
  // ...
  sendLoginOTP: (email) => post('/auth/send-login-otp', { email }, false),

// AFTER:
export const authAPI = {
  login: (email, password, captchaToken) => post('/auth/login', { email, password,
captchaToken }, false),
  // ...
  sendOTP: (email, name, captchaToken) => post('/auth/send-otp', { email, name, captchaToken
}, false),
  // ...
  sendLoginOTP: (email, captchaToken) => post('/auth/send-login-otp', { email, captchaToken
}, false),
```

File 4: `backend/controllers/authController.js`

4a — Add helper function at the very top of the file, after the `require` statements:

```
// ── Cloudflare Turnstile verification ──────────────────────────────
const verifyTurnstile = async (token, ip) => {
  try {
    const res = await fetch(
      'https://challenges.cloudflare.com/turnstile/v0/siteverify',
      {
        method:  'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          secret:   process.env.TURNSTILE_SECRET_KEY,
          response: token,
          remoteip: ip,
        }),
      }
    );
    const data = await res.json();
    return data.success === true;
  } catch (err) {
    console.error('Turnstile verify error:', err.message);
    return false;
  }
};
```

4b — Update `exports.login` (currently at line ~41) — add verification after the null check:

```
exports.login = async (req, res) => {
  try {
    const { email, password, captchaToken } = req.body;  // add captchaToken

    // Cloudflare Turnstile — web requests include token, Android requests skip
    if (captchaToken !== undefined) {
      const ok = await verifyTurnstile(captchaToken, req.ip);
      if (!ok) return res.status(403).json({ error: 'Security verification failed. Please
refresh and try again.' });
    }

    // ... rest of existing login logic unchanged (no other changes needed) ...
    if (!email || !password) { ... }
```

4c — Update `exports.sendOTP` — find the sendOTP handler and add the same check:

```
exports.sendOTP = async (req, res) => {
  try {
    const { email, name, captchaToken } = req.body;  // add captchaToken

    // Cloudflare Turnstile verification
    if (captchaToken !== undefined) {
      const ok = await verifyTurnstile(captchaToken, req.ip);
      if (!ok) return res.status(403).json({ error: 'Security verification failed. Please
refresh and try again.' });
```

```
    }

    // ... rest of existing sendOTP logic unchanged ...
```

4d — Update `exports.sendLoginOTP` — same pattern:

```
exports.sendLoginOTP = async (req, res) => {
  try {
    const { email, captchaToken } = req.body;  // add captchaToken

    // Cloudflare Turnstile verification
    if (captchaToken !== undefined) {
      const ok = await verifyTurnstile(captchaToken, req.ip);
      if (!ok) return res.status(403).json({ error: 'Security verification failed. Please
refresh and try again.' });
    }

    // ... rest of existing sendLoginOTP logic unchanged ...
```

Part 5 — Testing

Test in Development

1. Set `TURNSTILE_SECRET_KEY=1x0000000000000000000000000000000AA` in `backend/.env`
2. Set `data-sitekey="1x00000000000000000000AA"` in the widget JSX
3. Run `npm run dev:web` — the widget will render and auto-pass
4. Submit login/signup — should work exactly as before with no errors

Test Block Behaviour

1. Change `data-sitekey` to `2x00000000000000000000AB` (always blocks)
2. Change `TURNSTILE_SECRET_KEY` to `2x0000000000000000000000000000000AA`
3. Try to login — should get "Security verification failed" error
4. Revert both keys back to always-pass when done

Production Checklist Before Go-Live

- [ ] Replace test Site Key with real Site Key in `SplitAuthCard.js`
- [ ] Replace test Secret Key with real Secret Key in `backend/.env`
- [ ] Domain restricted to `skillsphere.com.pk` in Cloudflare dashboard
- [ ] `backend/.env` is in `.gitignore`
- [ ] Test login + signup on live site

Security Checklist

- ✅ Always verify on backend — frontend CAPTCHA alone is useless, bots POST directly to API
- ✅ Never put Secret Key in frontend — only Site Key goes in frontend code
- ✅ Restrict domain in Cloudflare dashboard — lock Site Key to `skillsphere.com.pk`
- ✅ Pass remoteip in verification — include `req.ip` for stronger protection
- ✅ Tokens are single-use — Cloudflare auto-invalidates after first successful verification

- ✅ Token expires in 5 minutes — the error callback resets the widget automatically
- ✅ Android is unaffected — `captchaToken` is `undefined` on Android, backend skips check

---

Files Changed Summary

File	Line	Change
`AppAndroidSS/web/index.html`	`<head>`	Add Turnstile `<script>` tag
`AppAndroidSS/src/screens/auth/SplitAuthCard.js`	~102	Add `captchaToken` state
`AppAndroidSS/src/screens/auth/SplitAuthCard.js`	~121	Register window callbacks in useEffect
`AppAndroidSS/src/screens/auth/SplitAuthCard.js`	~151	Update `handlePasswordLogin`
`AppAndroidSS/src/screens/auth/SplitAuthCard.js`	~159	Update `handleSendLoginOTP`
`AppAndroidSS/src/screens/auth/SplitAuthCard.js`	~202	Update `handleSignupSendOTP`
`AppAndroidSS/src/screens/auth/SplitAuthCard.js`	form JSX	Add `<div class="cf-turnstile">` widget
`AppAndroidSS/src/services/apiClient.js`	~200	Add `captchaToken` param to `login`, `sendOTP`, `sendLoginOTP`
`backend/controllers/authController.js`	top	Add `verifyTurnstile` helper function
`backend/controllers/authController.js`	`exports.login`	Add token verification
`backend/controllers/authController.js`	`exports.sendOTP`	Add token verification
`backend/controllers/authController.js`	`exports.sendLoginOTP`	Add token verification
`backend/.env`	—	Add `TURNSTILE_SECRET_KEY` variable

SkillSphere — Cloudflare Turnstile Setup Guide